GitHub Org Security & Access-Hygiene Scanner

Walks every repository in a GitHub organization and answers two questions a security lead otherwise has to answer by hand, repo by repo:

1. **Is this repo secure?** — every Dependabot advisory, by severity, state and age.
2. **Does everyone with access still deserve it?** — a contribution score per person per repo, and a ranked list of access grants that no longer look justified.

Output is a single self-contained HTML dashboard. **Access removal is always a suggestion. This tool never revokes anything.**

Try it in 30 seconds — no token, no org

```
python main.py --demo
```

This runs the complete pipeline against `fixtures/demo_org.json` (a synthetic 6-repo organization) and writes `out/dashboard.html`. The fixture pins its own clock, so the numbers are identical on every machine and every day — which makes it a stable reference when comparing your run to ours.

Don't want to run anything? A generated copy is committed at docs/demo_dashboard.html — open it straight from a clone. It is the output of the command above, and it contains only synthetic fixture data, which is why it is the one scan result in this repo that is safe to commit.

Regenerating it reproduces every finding, score and date exactly — the fixture pins its own clock, so the report does not drift. Two lines do differ: the run number and the "generated at" timestamp, which record when *your* run happened and are meant to vary. Diff the two files and those are the only changes you will see.

To turn it into a PDF, open it and use the browser's **Print** → **Save as PDF**. The page carries a print stylesheet that expands every collapsed repository section, unclips the wide tables and repeats table headers across pages.

See DESIGN_NOTES.md for why the demo exists and exactly what is mocked, and RUN_REPORT.md for a captured walkthrough of a real execution.

Documents

Start with the one-pager; the rest is there when you want the detail.

DOCUMENT	WHAT IT COVERS	PDF
SUBMISSION.md	One page: scoring logic and why, edge cases, production-readiness	pdf
README.md	Setup, token permissions, usage	pdf
DESIGN_NOTES.md	Every decision and its reasoning, what changed mid-build, AI-assistance disclosure	pdf

DOCUMENT	WHAT IT COVERS	PDF
RUN_REPORT.md	Step-by-step execution with captured output and dashboard screenshots — demo *and* a live organization	pdf
Dashboard (demo)	The generated report, from fixtures	html · pdf
Dashboard (live)	The same report from a real organization scan	html · pdf

Regenerate the whole bundle — PDFs and screenshots — from a clean clone:

```
python tools/make_docs.py
```

It renders through headless Chrome or Edge, so there is no extra toolchain to install, and the screenshots are produced from the committed dashboard rather than captured by hand, which keeps them from drifting.

Setup

1. Environment

Requires **Python 3.11+**.

```
python -m venv .venv
```

```
.venv\Scripts\activate
```

```
pip install -r requirements-dev.txt
```

On macOS/Linux the activate line is `source .venv/bin/activate`.

2. Token

Create a **fine-grained personal access token** scoped to the organization you want to scan.

SCOPE LEVEL	PERMISSION	ACCESS	USED FOR
Organization	Members	Read	org members and owners
Organization	Administration	Read	repository list
Organization	Dependabot alerts	Read	org-wide advisory scan (1 call)
Repository	Metadata	Read	repo details
Repository	Contents	Read	commit history
Repository	Pull requests	Read	PRs and reviews
Repository	Administration	Read	collaborators and permissions

A classic PAT with `repo`, `read:org` and `security_events` also works.

Without org-level *Dependabot alerts: read*, the scanner falls back to the per-repo endpoint automatically — slower, but it still works.

3. Configure

```
cp .env.example .env
```

Set at minimum:

```
GITHUB_TOKEN=github_pat_...
GITHUB_ORG=your-org-login
```

`.env` is gitignored. Every other setting has a working default; all of them live in config.py with the reasoning next to each value.

4. Run

```
python main.py
```

Then open `out/dashboard.html` in any browser. No server needed — Chart.js is inlined into the file.

Usage

COMMAND	WHAT IT DOES
`python main.py`	Full scan, then write the dashboard
`python main.py --demo`	Full pipeline on bundled fixtures; no token needed
`python main.py --resume`	Continue the last interrupted run instead of restarting
`python main.py --report-only`	Re-render the dashboard from SQLite; makes no API calls
`python main.py --json`	Print a machine-readable run summary to stdout
`python main.py --limit 5`	Scan at most 5 repos — good for a first smoke run
`python main.py --no-cache`	Ignore stored ETags and refetch everything

Useful combinations:

```
python main.py --limit 5 -v
```

```
python main.py --json > summary.json
```

The `--json` summary

Designed to be consumed by another tool — a CI job, a Slack notifier, a ticket opener:

```
{
  "run_id": 2,
  "org": "acme-demo",
  "mode": "demo",
  "status": "completed",
  "repos_scanned": 5,
  "repos_skipped": 1,
  "advisories_found": 10,
  "advisories_open": 8,
  "suggestions_made": 10,
  "members_excluded": 10,
  "top_suggestions": [
    {"login": "alex-departed", "repo": "acme-demo/payments-api",
     "permission": "admin", "risk": 3.0, "score": 0.0, "team_inherited": false}
  ],
  "dashboard": "out/dashboard.html",
  "api": {"requests": 412, "cache_hits": 190, "rate_limit_sleeps": 0}
}
```


Logs go to stderr, so `--json` output on stdout stays clean and pipeable.

How scoring works

```
activity = 3.0·ln(1+commits) + 2.5·ln(1+reviews)
         + 2.0·ln(1+PRs merged) + 1.0·ln(1+PRs opened)

score    = activity × 0.5^(days since last activity / 180)

risk     = permission weight / (1 + score)
```

Permission weights: `admin 3.0`, `maintain 2.0`, `write 1.5`, `triage 0.5`, `read 0.5`. A member is flagged when **score < 5.0**. Suggestions are sorted by risk, highest first.

For calibration: one recent commit alone scores 2.08, five score 5.37, ten score 7.19. So the default threshold flags roughly "*fewer than four recent commits and no review activity.*"

The reasoning behind every constant is in DESIGN_NOTES.md. To change them, edit `SCORING` in config.py or set the matching `.env` variables — nothing is hard-coded elsewhere.

Who is never flagged

Recorded with a reason and shown in section 4 of the dashboard, never silently dropped:

- **Org owners** — always, regardless of activity
- **Bots** — `type == "Bot"` or a login ending in `[bot]`
- **Allowlisted logins** — set `LOGIN_ALLOWLIST` in `.env`
- **Repos created inside the lookback window** — no time to build a history
- **Repos with only one contributor** — nothing to compare against

- **Empty repos** — no history at all

Forks are skipped entirely by default (`SKIP_FORKS=false` to include them). Archived repos are **scanned and tagged**, not skipped: stale admin access on an archived repo is still live access, and anyone holding it can unarchive the repo.

Tests

```
python -m pytest -q
```

156 tests covering the scoring arithmetic against hand-computed values, every exclusion rule, the full fixture organization end-to-end (including the exact expected ranking), database idempotency, the HTTP layer — rate-limit waits, `Retry-After`, ETag 304s, cursor pagination, GraphQL error handling — and the rule that a refused access listing is never rendered as "nobody has access", that an archived repo changes the remediation advice but never the score, that an organization's base permission is not mistaken for a team grant, and that a contributor holding no access is never suggested for removal.

CI runs the suite on Python 3.11 and 3.12, then runs `--demo` end-to-end and fails the build if the fixture findings drift from the numbers published in RUN_REPORT.md — see .github/workflows/tests.yml.

```
python -m pytest -q tests/test_score.py
```

Project layout

```
config.py      settings, weights, thresholds — every tunable in one file
client.py      HTTP: rate limits, ETag cache, backoff, typed errors
scan.py        repositories + advisories
contrib.py     collaborators (3 kinds) + contribution data via GraphQL
score.py       scoring, exclusions, recommendations — pure functions
report.py      dashboard rendering
db.py          SQLite schema and upserts
demo.py        fixture-backed demo mode
main.py        CLI entry point
fixtures/      synthetic org used by --demo and the tests
templates/     Jinja2 dashboard template
vendor/        Chart.js, inlined into the output so it works offline
tests/         pytest suite
docs/          committed demo dashboard, openable without running anything
.github/       CI, and a weekly scheduled scan that notifies but never revokes
```

Data lands in `data/scanner.sqlite3`; the dashboard in `out/dashboard.html`. Both are gitignored — scan results contain internal repo names and member logins.

Re-running is safe

Every table is keyed on natural GitHub IDs (`repo_id` , `(repo_id, ghsa_id, manifest_path)` , `(repo_id, login)`) and written with `INSERT ... ON CONFLICT DO UPDATE` . Running twice updates rows in place; it never duplicates them. Progress is committed per repo, so an interrupted run continues from where it stopped:

```
python main.py --resume
```

Stored ETags mean a re-run over an unchanged org is served largely from 304 responses, which do not count against the API quota.